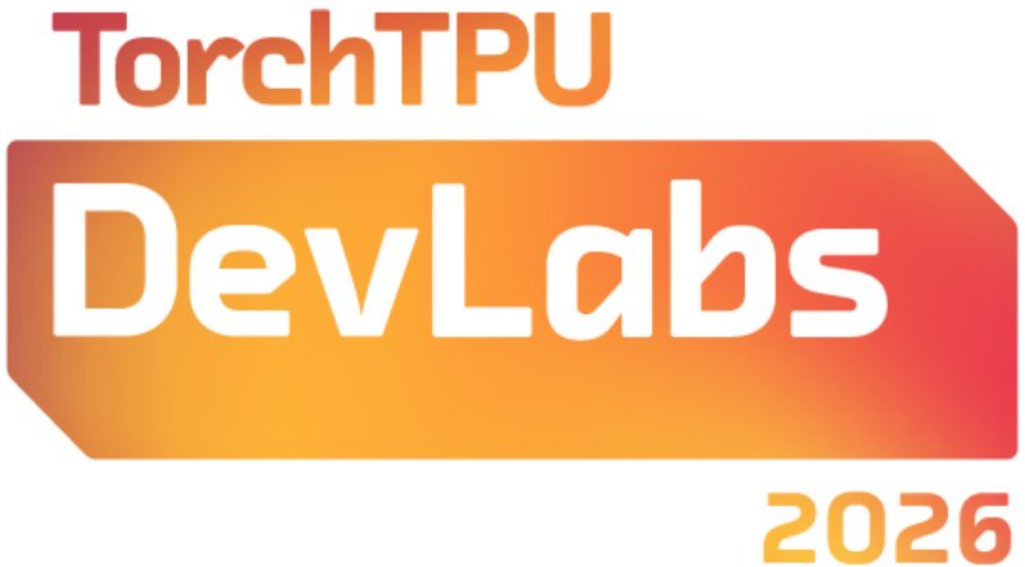
Inference with TorchTPU

TorchTPU DevLabs · Nov 9, 2026



Rob Mulla, Google
Developer Relations Engineer

THE NEXT THIRTY MINUTES

Running inference on TPUs with PyTorch

1

TorchTPU: the TPU is a PyTorch device

Set the device and run. **Demo 1**: one script on one chip, three ways: eager, DeferAndFuse, compiled.

2

Inference 101: what decides serving speed

The terms to know: attention and the KV cache, prefill and decode, HBM capacity and bandwidth. The picture, no demo.

3

vLLM and SGLang on TorchTPU

The engines you already use. **Demo 2**: SGLang, one prompt asked twice. **Demo 3**: vLLM, sixty-four requests at once.

Three live runs on real TPU VMs. Everything is open source.

TorchTPU makes the TPU a PyTorch device

PyTorch
your model, your code

→

TorchTPU
registers the `tpu` device and a `torch.compile` backend

→

Cloud TPU
one chip today; a slice of thousands the same way

the whole change

```
model = model.to("tpu")
model = torch.compile(model, backend="tpu")  # optional: for speed
```

Eager · the default

One op at a time

The PyTorch you know: print a tensor, set a breakpoint. Debug here.

DeferAndFuse · optimized eager

Ops deferred and fused

`TORCH_TPU_DEFER_AND_FUSE=1`: blocks of ops run together, up to a materialization point. No code change.

torch.compile · backend="tpu"

One program for the graph

Dynamo captures the model, TorchTPU compiles it once. The most speed.

Demo one runs the same script all three ways.

Demo 1. One script: eager, DeferAndFuse, compiled

Cloud TPU v6e-1, one Trillium chip

example*.py · same model, same loop · `device ="tpu"` · three ways to run it

PyTorch · eager · DeferAndFuse · torch.compile(backend="tpu")

TorchTPU



Cloud TPU v6e-1 · ~/demo · tpu-top on the right

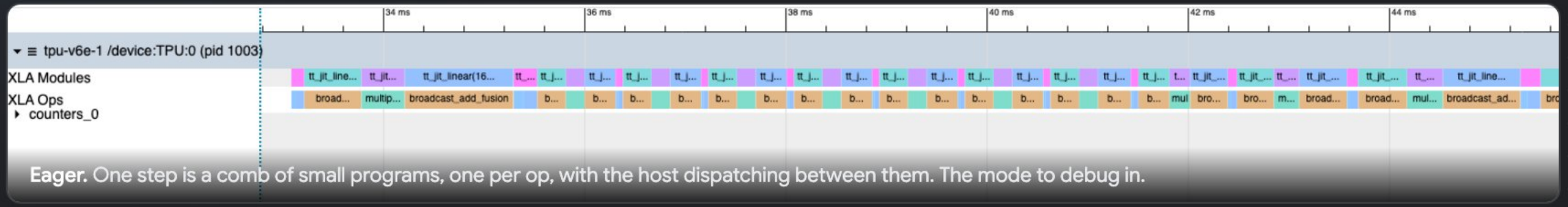
```
# 1. on the chip: device = "tpu", 1,000 steps
$ vim example1_eager.py
$ python example1_eager.py

# 2. the same file, one environment variable
$ TORCH_TPU_DEFER_AND_FUSE=1 python example1_eager.py

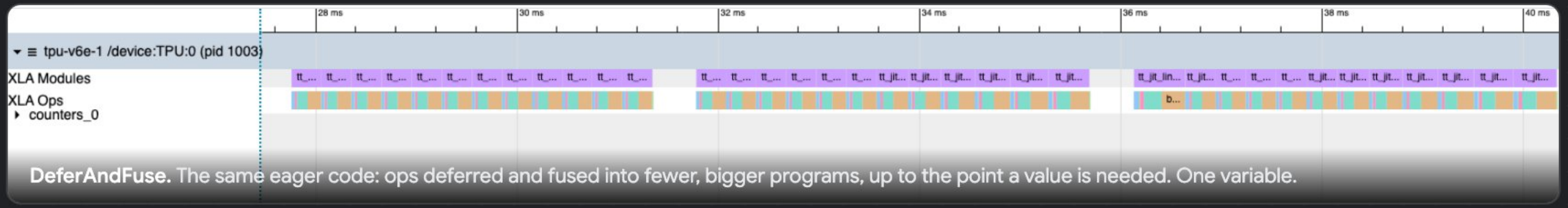
# 3. one more line: torch.compile(backend="tpu")
$ vim example2_compile.py
$ python example2_compile.py
```

● LIVE · switching to the terminal

One op at a time, fused blocks, or one program



Eager. One step is a comb of small programs, one per op, with the host dispatching between them. The mode to debug in.



DeferAndFuse. The same eager code: ops deferred and fused into fewer, bigger programs, up to the point a value is needed. One variable.



Compiled. The same step is one program, the ops fused inside it, back to back with the next step. The mode to ship.

The standard PyTorch profiler with a TPU config, on the scripts you just watched. Same scale on all three.

Every new token looks at every token before it



THE NEW TOKEN

QUERY

compare the query with every key: one score per earlier token

add up the values, weighted by those scores: the output for this token

`attention · a matrix multiply per step`

KV cache

the keys and values of every token so far

Earlier tokens' keys and values never change, so the engine keeps them. Without the cache every new token would recompute all the ones before it; with it, each step computes only its own, and reads the rest.

Serving a model is two phases

Prefill

THE PROMPT



MATRIX UNITS

The whole prompt goes through the model at once. Compute-bound.

Heavy when prompts are long: summarising a document, a codebase in context, RAG.

compute-bound

Decode

HBM: WEIGHTS AND KV CACHE

THE ANSWER



One token a step, and every step re-reads the weights and the cache. Memory-bound.

Heavy when answers are long: chat, code generation, reasoning, agents.

memory-bound

In a conversation the prompt was mostly seen before, so with the cache kept, prefill shrinks to the new tokens and what remains is decode: memory-bound. Serving is a memory problem.

Two numbers decide most of serving

Capacity

A 32B MODEL IN BFLOAT16 ON FOUR TRILLIUM CHIPS

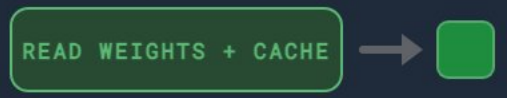


Does the model fit, and what is left for the cache? Too big for one chip: a slice of every layer on each of four, linked over ICI.

`--tensor-parallel-size 4`

Bandwidth

ONE USER, ONE STEP



EIGHT USERS IN ONE BATCH, ONE STEP

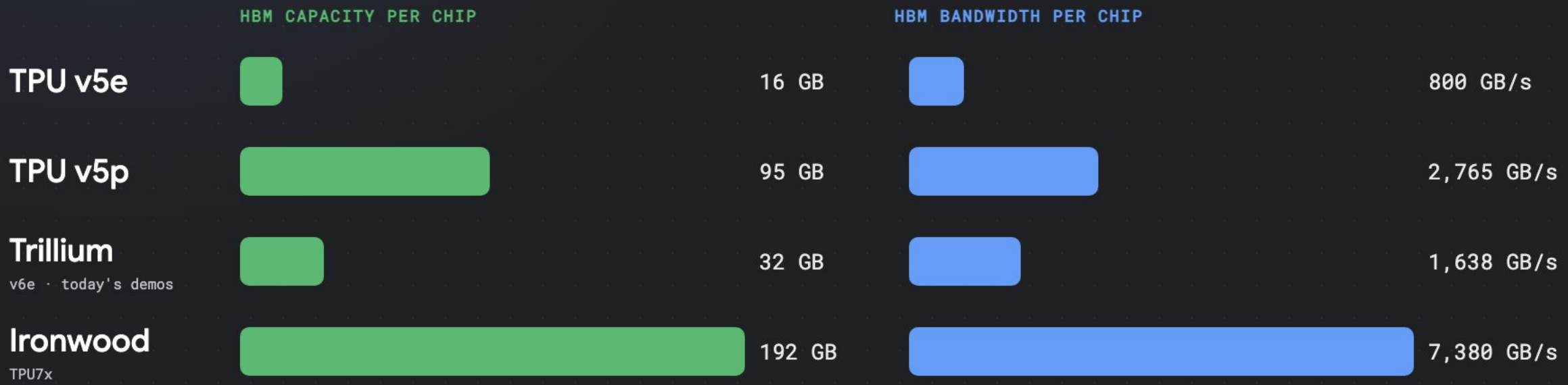


One user: a step reads the weights and the cache for one token, so bandwidth is that user's speed. A batch: the same read serves everyone in the step. Batching is throughput.

`one user's speed · everyone's throughput`

Every lever in the rest of the half hour moves one of these two numbers.

Cloud TPU HBM: capacity and bandwidth by generation



TPU 8i
announced

"TPU 8i is built for large-scale inference and reinforcement learning."

Google Cloud blog, TPU 8t and TPU 8i technical deep dive, Apr 22, 2026 · its table: 288 GB of HBM, 8,601 GB/s

Capacity decides what fits next to the KV cache; bandwidth decides how fast tokens come out. Figures: the Cloud TPU documentation's specification tables.

Why use an engine?

Agents and multi-turn

Prefix caching demo 2 · the cache kept in host memory · tool calling

Many users at once

Continuous batching demo 3 · speculative decoding · structured output

Big models, long contexts

Quantized checkpoints · a compact FP8 KV cache · long prompts split across chips

Many chips, many hosts

Every kind of parallelism · prefill and decode on separate hosts · multimodal input

Supported today or being added, in both engines. The fleet layer, GKE with llm-d: tomorrow 1 PM, Newton Jain.

Install and serve the way you already do

vLLM · vllm-torchtpu
built with Inferact, founded by the creators of vLLM

SGLang · sglang-torchtpu
built with RadixArk, founded by the creators of SGLang

install, then serve

```
$ pip install vllm-torchtpu

$ vllm serve Qwen/Qwen3-32B \
    --tensor-parallel-size 4

# no TPU flag: the plugin registers itself
```

install, then serve

```
$ pip install sglang-*.whl sglang_tpu-*.whl
# the two release wheels of sglang-torchtpu
$ python -m sglang.launch_server \
    --model-path Qwen/Qwen3-32B --device tpu \
    --tp-size 4 --enable-torch-compile
```

Same flags, same OpenAI-compatible API. The TPU work is underneath, in kernels the engines ship.

Both open source, both on TorchTPU.

Where the engines sit: the TPU inference stack

Every layer is open source. The chips at the bottom, your application at the top; the engines in the middle.

AGENTS AND APPLICATIONS	your chat, your coding tool, your agent sandbox	
ORCHESTRATION AND CONTROL	GKE · llm-d · Ray	TOMORROW, 1 PM: NEWTON
INFERENCE ENGINES	vLLM · SGLang · vllm-torchtpu · sglang-torchtpu	DEMOS 2 AND 3
KV CACHE MANAGEMENT	TPU Sync · Mooncake Store · LMCache · HiCache	
FRAMEWORK, COMPILER, KERNELS	PyTorch · TorchTPU · Pallas · OpenXLA	DEMO 1
TPU · STORAGE · NETWORK	Trillium, Ironwood · DRAM, Lustre · ICI, DCN	

Paged attention, and the ragged version TPUs use

requests

A: What | is | a | KV | cache | ? | It | keeps | past | keys

B: Name | three | TPUs

C: What | is | a | KV | page | ?

C starts with A's text. Its first page is A's page, shared, not copied. Only its two new tokens are computed. Prefix caching: next slide.

Paged: fixed pages and a table per request, like an operating system's virtual memory. Nothing reserved, nothing wasted.

Ragged, on a TPU: fixed-shape programs, so the mixed batch is one kernel over the pages, in fixed tiles, no padding.

Shared: a prompt that starts like an earlier one points at its pages instead of recomputing them.

Simplified: one entry per token stands for one per layer; six tokens for thousands; pages of four for the engines' pages of 128.

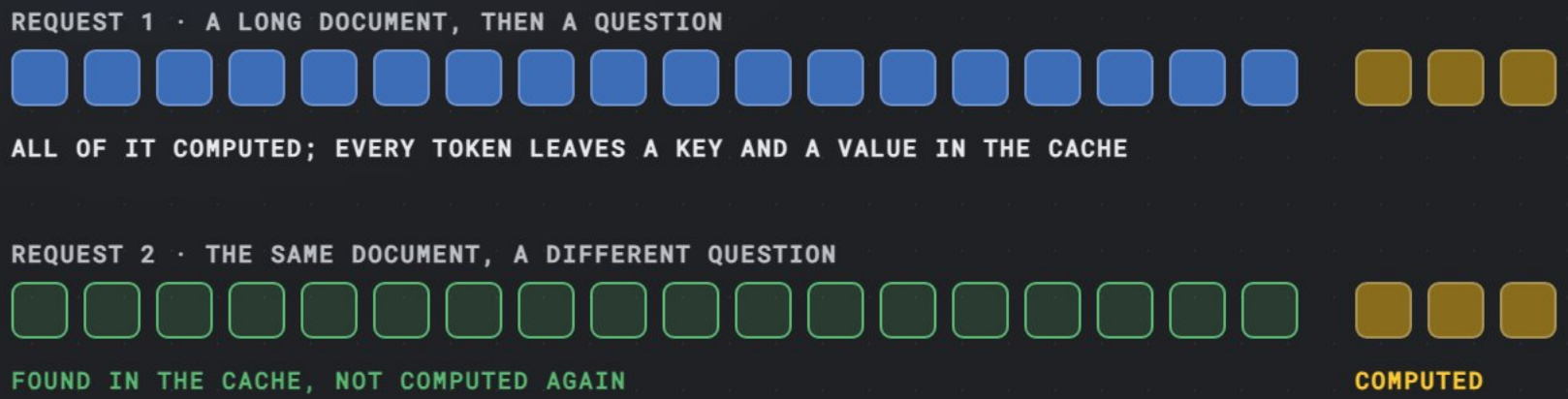


this step

matrix units busy

HBM read per token

Prefix caching: ask about the same document twice, pay once



KV CACHE

kept after the request, keyed by the tokens

Both engines keep the pages after a request and point the next request at them when it starts the same way. A pointer, not a copy. On by default.

Demo 2. One long prompt, asked about twice

SGLang on TorchTPU · Cloud TPU v6e-4, four Trillium chips

ask.py · one long prompt, two questions — :30000/v1

SGL Qwen3-32B · four chips · prefix caching on

TorchTPU

Cloud TPU v6e-4 · one VM, four Trillium chips



Qwen3-32B split four ways · --tp-size 4

Cloud TPU v6e-4 · ~/demo · tpu-top on the right

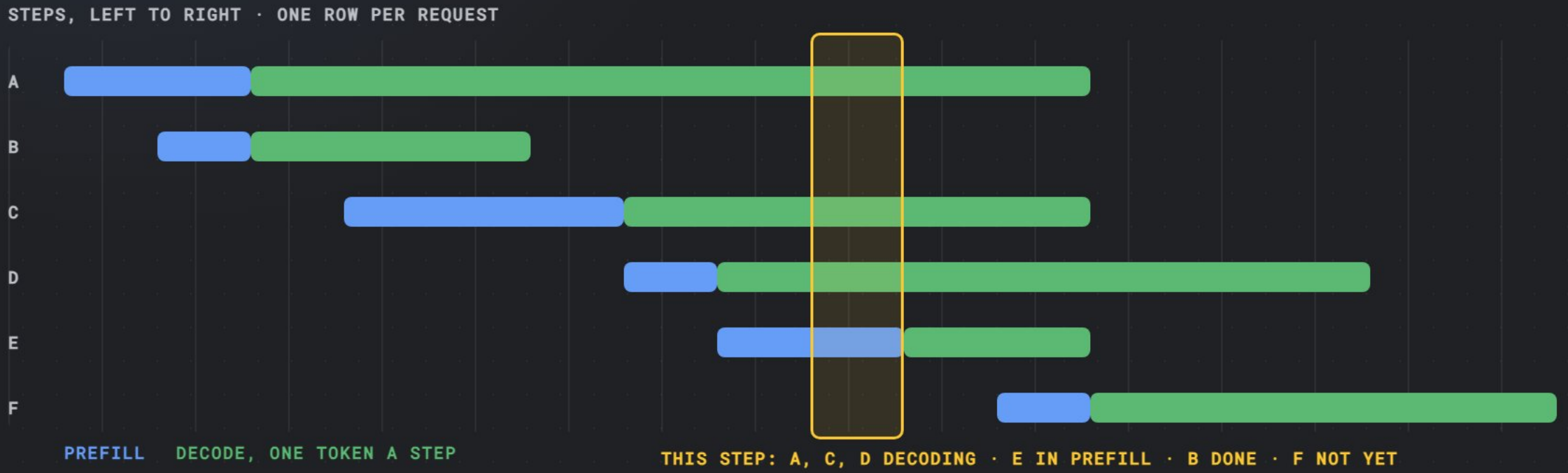
```
# how the server was started, before the session
$ vim serve.sh
    python -m sglang.launch_server
      --model-path Qwen/Qwen3-32B --device tpu
      --tp-size 4 ... --enable-cache-report

# the client: an ~11k-token prompt, two questions
$ vim ask.py
$ python ask.py

# watch: first token ... ms, cached N of M tokens
```

● LIVE · switching to the terminal

Continuous batching: requests join and leave every step



The batch is re-formed every step: a request joins as soon as it arrives and leaves as soon as it finishes. One weight read serves everyone in the step.

`--max-num-seqs 128`

Demo 3 sends sixty-four requests of different lengths at once and watches them stream together.

Demo 3. One request, then 64 at once

vLLM on TorchTPU · Cloud TPU v6e-4, four Trillium chips

burst.py · 1 request, then 64 at once — :8000/v1

vLLM Qwen3-32B · four chips · Ragged Paged Attention

TorchTPU

Cloud TPU v6e-4 · one VM, four Trillium chips



Qwen3-32B split four ways · --tensor-parallel-size 4

Cloud TPU v6e-4 · ~/demo · tpu-top on the right

```
# how the server was started, before the session
$ vim serve.sh
    vllm serve Qwen/Qwen3-32B
      --tensor-parallel-size 4 --max-num-seqs 128 ...

# the client: n requests at once, one bar each
$ vim burst.py
$ python burst.py 1
$ python burst.py 64

# watch: tokens/s, all four chips busy in tpu-top
```

● LIVE · switching to the terminal

Three things to take home

1

TorchTPU: the TPU is a PyTorch device.

Set the device and run. Debug in eager; `TORCH_TPU_DEFER_AND_FUSE=1` for speed with no code change; `torch.compile` for the most. The profiler shows you which one you have.

2

Inference 101: serving speed is a memory story.

Every token reads the KV cache; prefill is compute, decode is memory; HBM capacity and bandwidth decide what fits and how fast, and batching turns bandwidth into throughput.

3

vLLM and SGLang on TorchTPU.

Install and run the way you know; paged attention, prefix caching and continuous batching work the same, and the TPU work is underneath, in kernels they ship.

DO IT YOURSELF

Try it yourself: in Colab now, or on a TPU VM

In Colab, now

Three notebooks, one chip, a small model (Qwen3-0.6B).

01 · TORCHTPU ON ONE CHIP
device line, eager, compile, profiles side by side

02 · SGLANG-TORCHTPU
launch_server --device tpu, a burst, the prefix cache

03 · VLLM-TORCHTPU
vllm serve, the same with vLLM

`Open in Colab, from the README`

On a TPU VM

The three demos as you saw them, with the exact commands.

DEMO 1 · ONE CHIP
example*.py · profile_*.py

DEMOS 2 AND 3 · FOUR CHIPS
serve.sh · ask.py · burst.py, Qwen3-32B

THE ENGINES' RECIPES
vLLM docs/deployment · SGLang benchmark/tpu

`two flags · --tensor-parallel-size --enable-torch-compile (SGLang)`

github.com/RobMulla/torchtpu-inference-workshop

github.com/google-pytorch/torch_tpu
github.com/vllm-project/vllm-torchtpu
github.com/sgl-project/sglang-torchtpu
docs.cloud.google.com/tpu/docs/pytorch-on-tpu

Questions now, or find me after **the terminal stays open**